Backtesting System Architecture

Oct 2, 2026 · @qiwen

Overview

The system has 8 layers that talk to each other only through events. It targets mid- to low-frequency long-options strategies (holding periods of days to weeks), while the core stays asset-agnostic so stocks, ETFs and futures run on the same engine.

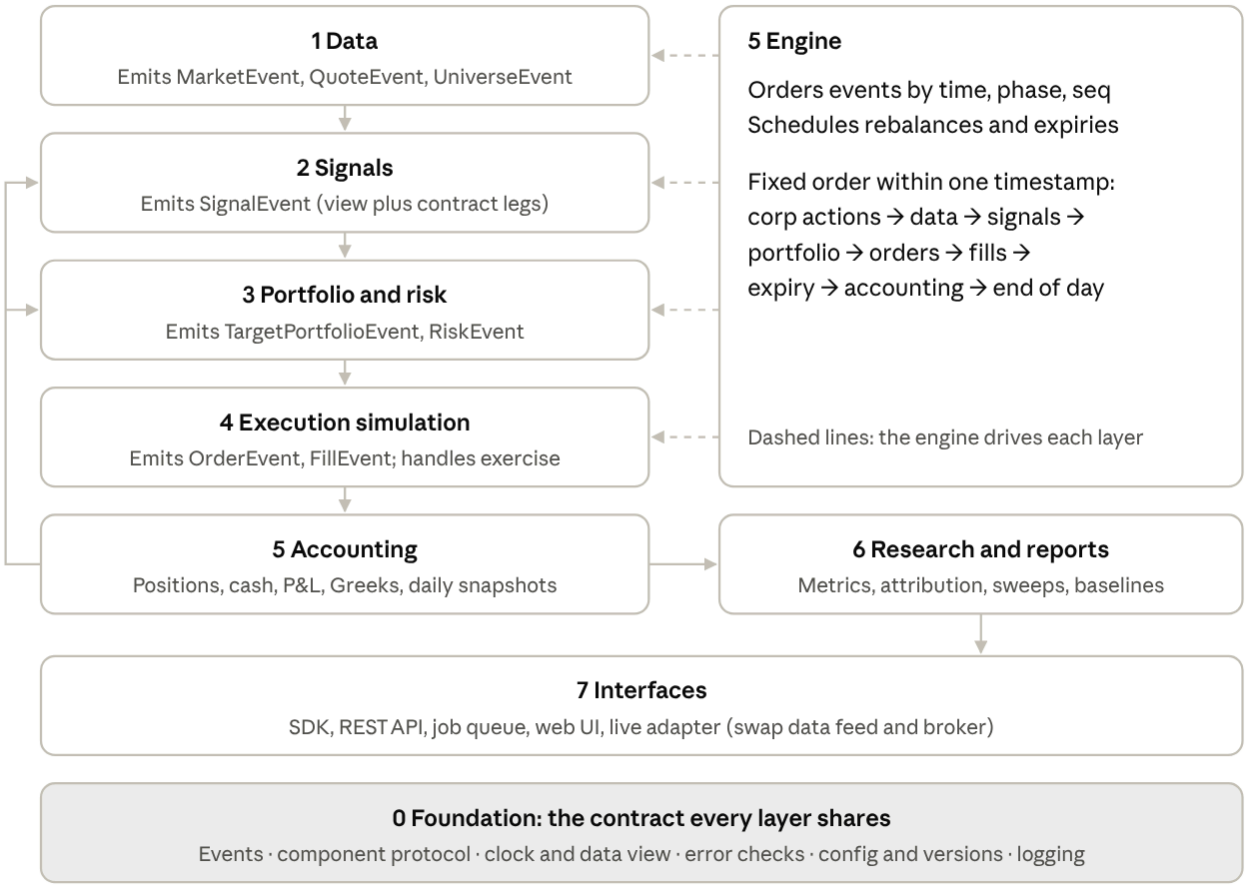
Five principles apply to every layer:

1. One-way dependencies: a layer depends only on interfaces defined below it.
2. Correctness by mechanism: look-ahead, illegal trades and accounting errors are blocked by the foundation, not by the strategy author's discipline.
3. Same code for backtest and live: strategy, portfolio and risk components run unchanged in both.
4. Pessimistic by default: fills, costs and capacity err toward understating returns. For options, that means paying the spread.
5. Reproducible: config, code version and data version uniquely determine a run's result.

Options differ from stocks in three ways, and each layer below reflects them:

- The tradable set changes every day. Contracts list and expire, so the universe is an option chain, not a ticker list.
- Price has more dimensions. Spot, implied volatility and time all move the price, so risk is measured in Greeks.
- Costs are dominated by the bid-ask spread and time decay, not by commissions.

Events flow from data to accounts; positions feed back to decisions

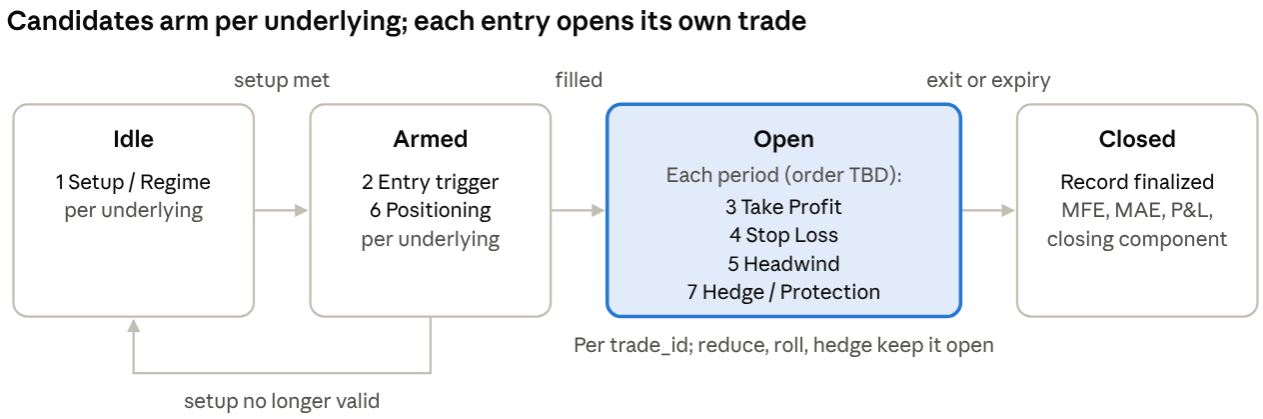


System architecture · 8 layers, event-driven

The main path runs top-down: data → signals → portfolio and risk → execution → accounting. Positions and cash flow back read-only to the signal and portfolio layers, and the engine drives the processing order.

Strategy Model: Seven Components

The strategy is defined by seven components that follow a trade from setup to exit. This section fixes only their structure: where each one sits, when it runs, what it can read and what it can emit. The internal logic and form of each component are not decided yet; each is an empty slot that any variant can fill and that can be researched, swapped or tuned on its own.



Trade lifecycle · 4 states, 7 components

A trade waits in Idle until Setup / Regime arms it, opens when Entry fires and Positioning sizes it, and is managed each period by Take Profit, Stop Loss, Headwind and Hedge until it closes. The order in which those four run is a lifecycle setting, not yet decided.

Lifecycle states

State	Scope	Entered when	Component slots that run
Idle	Candidate (one per underlying)	No valid setup	1 Setup / Regime
Armed	Candidate (one per underlying)	Setup / Regime marks the candidate as ready; returns to Idle when the setup is no longer valid	1 Setup / Regime, 2 Entry, then 6 Positioning when Entry fires
Open	Trade (one per trade_id)	First fill received; the candidate's setup context is copied into the new trade	3 Take Profit, 4 Stop Loss, 5 Headwind, 7 Hedge / Protection, every snapshot
Closed	Trade (one per trade_id)	Position fully exited or expired	None; the record is finalized and shadow tracking starts (see 5.2)

Setups belong to the underlying and trades belong to a trade_id. One candidate can therefore produce several trades over time, and whether it may hold more than one at once is a lifecycle setting.

Lifecycle settings are left open on purpose. They live in the lifecycle config, not inside any component, so they can be researched like the components themselves:

Setting	Question it answers
open_state_order	In what order do slots 3, 4, 5 and 7 run each snapshot?
conflict_policy	How are conflicting actions in the same snapshot resolved?
max_concurrent_trades	How many open trades may one underlying hold at once?
reentry_policy	When may a candidate arm again after a trade closes?
check_frequency	At which snapshots do the Open-state slots run? It must match live trading exactly; no intra-snapshot stop or target fills are assumed

Component contracts

The table below is the default wiring for each slot: the level it works at, when it is evaluated, the data it reads, the actions it can emit and what it writes to the context. These are defaults to start from; any of them can be changed, and what a slot does with its inputs is left to its variants.

Slot	Level	Evaluated	Reads (default)	Emits	Writes to context
1 Setup / Regime	Underlying (candidate)	Each snapshot while Idle or Armed	Underlying price history (completed bars plus the current snapshot), realized volatility, ATM IV, IV rank (rolling), term structure, skew, event calendar, regime features	Arm or disarm the candidate	State, reference levels, score, regime label
2 Entry	Underlying, then contracts	Each snapshot while Armed	Underlying snapshot, setup context, option chain (quotes, IV, Greeks, spread, open interest, days to expiry)	OPEN with legs, or nothing	Trigger, reference price, IV and Greeks at entry

Slot	Level	Evaluated	Reads (default)	Emits	Writes to context
6 Positioning	Trade within the book	When OPEN or ADD is issued, or REDUCE needs a quantity	Setup score, entry context, premium budget, book premium at risk, book Greeks, liquidity caps (displayed size, volume)	Contract quantities	Quantity, budget used
3 Take Profit	Trade	Each snapshot while Open	Premium P&L, underlying versus setup and entry reference levels, MFE, days to expiry, Greeks, IV change since entry	REDUCE, CLOSE, SET_STOP, request HEDGE_ADD, HOLD	Levels reached
4 Stop Loss	Trade	Each snapshot while Open	Premium P&L, underlying versus setup and entry reference levels, MAE, time in trade, IV change	CLOSE, REDUCE, HOLD	Active stop level, trigger
5 Headwind (trade)	Trade	Each snapshot while Open	Changes since entry: regime label, IV, theta and days to expiry, delta, spread and liquidity, upcoming events; other slots' context	REDUCE, CLOSE, ROLL, SET_STOP, request HEDGE_ADD, HOLD	Active factors, severity
5 Headwind (book)	Book (Layer 3)	Each snapshot	Book Greeks, premium at risk, drawdown, market-wide regime and volatility	Scale, veto or force actions on trades	Outcome written to each affected trade

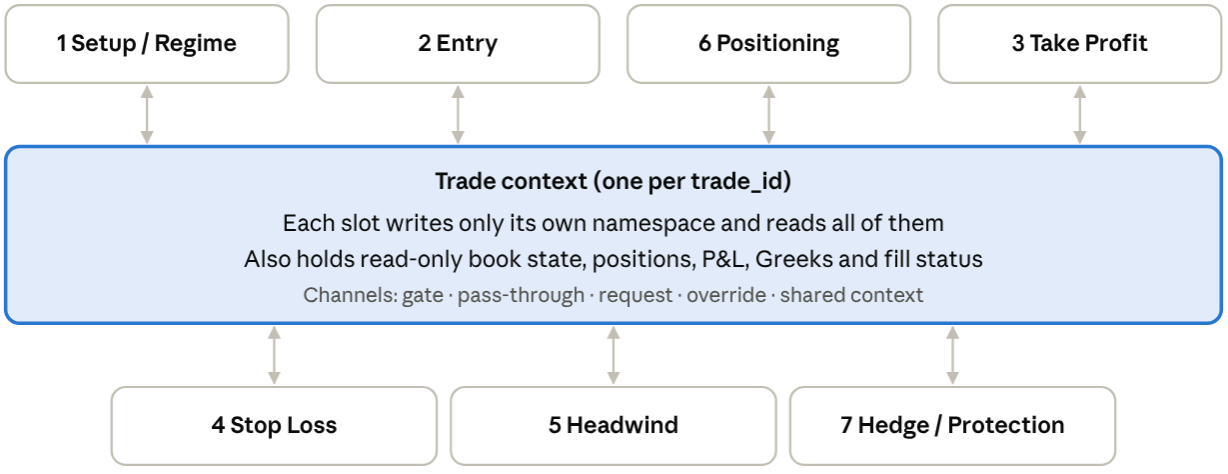
Slot	Level	Evaluated	Reads (default)	Emits	Writes to context
7 Hedge (trade)	Trade	On request, or each snapshot while Open	Trade Greeks and P&L, headwind context, quotes and cost of hedge instruments (underlying shares, index futures, options)	HEDGE_ADD, HEDGE_REMOVE	Hedge legs, hedge cost
7 Hedge (book)	Book (Layer 3)	Each snapshot	Book Greeks, scenario losses, hedge instrument quotes	Hedge legs under a book-level trade_id	Book hedge record

Every slot reads only through the DataView, so the available_at cut applies to all of them. A variant may read less than its default, or more within the DataView.

Component interactions

Slots never call each other directly. Every influence between them, and between them and the rest of the system, goes through two structures: a shared trade context and trade actions. Which links a strategy actually uses is decided by its variants, not by the architecture.

Slots influence each other only through the trade context and actions



Component interactions · 7 slots, 1 shared trade context

The trade context is a per-trade_id record. Setup / Regime writes to a candidate context per underlying, which is copied into the trade context when the trade opens. Each slot writes only to its own namespace (`setup` , `entry` , `take_profit` , `stop_loss` , `headwind` ,

`positioning`, `hedge`) and can read all of them, plus read-only book, position and fill information. Every write is timestamped and logged, so research can trace which information reached which decision.

Influence channels between slots

Channel	Meaning	Mechanism	Possible links (decided per variant)
Gate	One slot decides whether another may run	Lifecycle state	Setup / Regime → Entry
Pass-through	One slot's output becomes another's input	Written to the trade context, read by later slots	Setup or Entry → Positioning, Take Profit, Stop Loss, Headwind, Hedge
Request	One slot asks another to act	A trade action handled by the owning slot (for example HEDGE_ADD by Hedge, ADD or REDUCE quantities by Positioning)	Headwind → Hedge, Stop Loss, Positioning; Take Profit → Hedge
Override	Two slots issue conflicting actions in the same period	Conflict policy in the lifecycle config	Any of the Open-state slots
Shared context	Market state visible to every slot	Regime and other features in the DataView	Setup / Regime features → any slot

Links between slots and the rest of the system

Direction	Through	What it changes
Slots → Signals output	Trade actions → SignalCombiner → SignalEvent	What the strategy wants to do this period
Portfolio and risk → slots	Constraint, RiskRule, book-wide Headwind and Hedge	Scale, veto or force actions; the outcome (for example "vetoed by premium limit") is written back to the trade context so slots can see it

Direction	Through	What it changes
Execution → slots	Fill and order status	Trade context records fills, partial fills and rejects; lifecycle state changes only on fills (Armed → Open on the first fill)
Accounting → slots	Positions, P&L, Greeks, MFE and MAE	What Open-state slots read each period
Data → slots	DataView, cut at available_at	All market information a slot can use
Engine → slots	Lifecycle state, period timer, expiry events	When each slot runs

Because slots can depend on each other through these channels, researching one slot in isolation is not always enough. Section 6.3 adds interaction tests for that.

Two practical limits on Hedge: if a variant sells an option leg, Layers 4 and 5 need margin and assignment modeling first; and in the US, delta hedging can use the underlying shares (short sales need a borrow), index futures such as ES and MES, or options, so Layers 4 and 5 must support whichever instruments are chosen.

Action vocabulary

Every component emits TradeAction events from one shared vocabulary, so all decisions can be logged, compared and attributed the same way:

Action	Meaning
OPEN	Open a new trade with its legs
ADD	Increase an open trade (scale in)
REDUCE	Close a fraction of the trade
CLOSE	Close the whole trade, hedge legs included
ROLL	Replace legs with a later expiry or new strike
HEDGE_ADD / HEDGE_REMOVE	Attach or remove protective legs
SET_STOP	Move the stop level
HOLD	Explicit no-change, logged with the reason

Each action carries trade_id, component_id, variant version, reason and the data values that triggered it.

Research hooks per component

Component	Core question	Research tools that apply (see 6.3)
1 Setup / Regime	Does gating entries improve results, and in which conditions?	Ablation, variant swap, results split by setup state
2 Entry	Is the entry timing better than random within valid setups?	Random-entry baseline, variant swap, event study
3 Take Profit	Does it lock in gains or cut winners too early?	Counterfactual exits, variant swap, MFE analysis
4 Stop Loss	Does it save more than it costs?	Counterfactual exits, variant swap, MAE analysis
5 Headwind	Which responses to adverse factors help?	Ablation per factor, counterfactual exits, attribution by component
6 Positioning	Does sizing add value beyond trade selection?	Flat-size baseline, variant swap, drawdown comparison
7 Hedge / Protection	Is protection worth its cost?	Ablation, hedge-leg P&L, tail and drawdown comparison

Every trade records MFE and MAE (maximum favorable and adverse excursion) in both premium and underlying terms. They are the raw material for Take Profit and Stop Loss research.

Modifying a component

- Every variant is registered by name and version; the config chooses which variant fills each slot and its parameters.
- Changing one slot produces a run that differs from its baseline in that slot alone, so the two results are directly comparable.
- Each variant ships with unit tests on fixed scenarios, so a modified variant cannot silently change behavior elsewhere.

Config skeleton (slots only; variants to be defined):

```
strategy:
  setup:        {variant: tbd, version: tbd, params: {}}
  entry:        {variant: tbd, version: tbd, params: {}}
  take_profit:  {variant: tbd, version: tbd, params: {}}
  stop_loss:    {variant: tbd, version: tbd, params: {}}
  headwind:     {variant: tbd, version: tbd, params: {}}
  positioning:  {variant: tbd, version: tbd, params: {}}
  hedge:        {variant: tbd, version: tbd, params: {}}
  lifecycle:
    open_state_order: tbd        # run order of slots 3, 4, 5, 7
    conflict_policy: tbd         # how same-snapshot conflicts resolve
    max_concurrent_trades: tbd   # per underlying
    reentry_policy: tbd          # when a candidate may arm again
    check_frequency: tbd         # must equal live trading
  timing:
    signal_snapshot: tbd         # e.g. 15:45 ET
    fill_latency: tbd            # e.g. next quote 5 minutes later
```

Layer 0: Foundation

The foundation defines the contract every layer shares: events, the component interface, time and data visibility, and correctness checks. Fix it first, and the layers above can be built and swapped independently.

0.1 Events

Events are the only way layers communicate. Every event is immutable and carries a timestamp `ts` and a sequence number `seq` for ordering within the same instant.

Event	Producer	Consumers	Key fields
MarketEvent (bar)	Data	Signals, Execution, Accounting	symbol, ts, OHLCV, available_at
QuoteEvent (snapshot)	Data	Signals, Execution, Accounting	symbol, ts, bid, ask, bid/ask size, last, volume, open interest, available_at
CorporateActionEvent	Data	Accounting	type (dividend, split, rights), ex-date, ratio
ContractAdjustEvent	Data	Accounting, Signals	contract, old and new multiplier, old and new strike, new code

Event	Producer	Consumers	Key fields
UniverseEvent	Data	Signals	effective date, members (underlyings and listed contracts)
TimerEvent	Clock	All layers	start of day, snapshot time, rebalance day, end of day
SignalEvent	Signals	Portfolio	trade_id, action (OPEN, REDUCE, CLOSE, ROLL, HEDGE_ADD and others), underlying, legs, fraction, reason, component_id
TargetPortfolioEvent	Portfolio	Execution	target quantity per contract, reasons
RiskEvent	Portfolio risk	Portfolio, Engine	triggered rule, action (scale down, no new positions, liquidate)
OrderEvent	Execution	Matching	order_id, side, quantity, type, limit price, time in force
FillEvent	Matching	Accounting	filled quantity, price, fee, slippage, ts
OrderStatusEvent	Matching	Portfolio, Logging	partial fill, reject, cancel, and reason
ExpiryEvent	Clock	Accounting, Signals	contract, settlement price, exercise or lapse decision

Processing order within one timestamp is fixed: corporate actions and contract adjustments → market data → signals → portfolio and risk → orders → fills → expiry settlement → accounting → end-of-day snapshot.

0.2 Component protocol

All replaceable logic implements one base class. A component reads context and affects the system only by returning events. It never mutates the account directly.

```
class Component:
    name: str                 # registry name, referenced from config
    params: ParamSchema       # types, defaults, valid ranges

    def on_start(self, ctx): ...
    def on_event(self, event, ctx) -> list[Event]: ...
    def on_end(self, ctx): ...
    def get_state(self) -> dict: ...   # serializable, for resume
    def set_state(self, state): ...
```

`ctx` provides `clock` (current time and calendars), `data` (the data view), `portfolio` (positions, cash and Greeks, read-only), `logger` and `config`.

Layer	Component subtypes
Data	Loader, Validator, Adjuster, FeatureCalculator, RegimeClassifier, IVCalculator, GreeksCalculator, SurfaceBuilder
Signals	UniverseSelector, Filter, SetupRule, EntryRule, ContractSelector, StructureBuilder, TakeProfitRule, StopLossRule, HeadwindRule, PositionHedgeRule, RollRule, SignalCombiner
Portfolio and risk	RebalanceRule, Sizer (Positioning), Optimizer, Constraint, RiskRule, ScenarioRule, PortfolioHedgeRule
Execution	MarketRule, FillModel, SlippageModel, CostModel, ExerciseModel

Components register by name. The parameter schema drives both config validation and auto-generated web forms.

0.3 Clock and data view

- Clock: trading calendar (holidays, half days), session phases (opening cross, continuous trading, closing cross) in US Eastern time and the expiry calendar. It is the single source of `now`.
- DataView(t): returns only data with `available_at ⩽ t`. Queries: `history(symbol, field, n)`, `cross_section(field)`, `panel(fields, window)`, plus `chain(underlying, filters)` for listed contracts with quotes, IV and Greeks, `surface(underlying)` for the IV surface, and `contract(symbol)` for static terms. Out-of-range access raises instead of returning silently.

- Timing rule: the relation between signal time and fill time is set once in config and strategies cannot override it. For options, use fixed intraday snapshots: the signal at snapshot t sees completed daily bars plus the snapshot itself, never that day's close, and the fill uses a later quote at t plus a decision latency (for example signal at 15:45 ET, fill at 15:50 ET). Filling on the same quote the signal was computed from is a subtle form of look-ahead.

0.4 Error prevention

Risk	Mechanism	Where
Look-ahead bias	DataView cuts at available_at; delay test	Data view, Research
Survivorship bias	Historical constituents, delisted and expired contracts included	Data
Bad data	Price jumps, zero volume, gaps, duplicate timestamps	Data validation
Stale or out-of-sync quotes	Option and underlying quotes must fall within a set time tolerance; stale quotes are flagged	Data validation
Mid-price fills	Buys fill at ask or worse, sells at bid or worse	Execution
Accounting errors	Invariant assertions on cash, positions, equity and daily P&L	Accounting
Illegal trades	Orders breaking trading hours, halts, position limits or borrow availability are rejected	Execution
Non-reproducibility	Fixed event order and random seeds; result hash comparison	Engine
Regressions	Golden strategies (buy-and-hold, equal-weight rebalance, a single long call held to expiry) locked	Tests and CI

Invariants checked at minimum: cash ⩾ 0 for non-margin accounts; position quantity equals cumulative fills; equity equals cash plus position value at the current multiplier; daily P&L equals equity change minus external flows; no open position in an expired contract.

0.5 Configuration and versioning

Four things uniquely define a run: strategy config (components and parameters), code commit, data snapshot ID and run settings (period, initial capital, cost model). Their hash forms the `run_id`.

Config is YAML, validated against a schema (for example pydantic) at load time. Unknown fields raise errors.

0.6 Logging and traceability

Every fill can be traced back through fill → order → target → signal → component, along with the data values used at the time (including the IV and Greeks at entry). Logs have three levels: decision log (on by default), full event stream (for debugging), and errors and warnings.

Layer 1: Data

The data layer turns raw data into clean data that was truly available at time t. Most of a backtest's credibility is decided here, and for options most of the extra work is here too.

Pipeline: ingest → validate → clean → adjust → point-in-time alignment → IV and Greeks → surface → universe → features → storage and access.

Submodule	Responsibility	Details
Ingest	Pull raw data from sources	Underlying bars, option quotes, contract terms, corporate actions, index constituents, calendars, trading halts, earnings and event calendars, rates, dividends; incremental updates with retries; raw data is append-only
ContractMaster	Static terms of every contract	Underlying, call or put, strike, expiry, multiplier, exercise style, listing and delisting dates, full adjustment history

Submodule	Responsibility	Details
Quotes	Tradable prices	Option bid, ask and sizes at a fixed intraday snapshot plus volume, open interest and settlement price; the underlying quote at the same timestamp
Rates and dividends	Pricing inputs	Treasury or SOFR curve by tenor; discrete dividend schedule with ex-dates (needed to price American calls and judge early exercise); implied forwards from put-call parity, approximate for American options
Validate	Find problems, never fix them silently	Gaps, duplicate timestamps, OHLC contradictions, price jumps, zero volume; for options also stale quotes, crossed or locked markets, very wide spreads, prices below intrinsic value; a quality report per update
Clean	Apply explicit rules	Every rule configurable and logged; suspended or quote-less contracts are marked untradable, never forward-filled
Adjust	Corporate actions and contract adjustments	Store raw prices plus adjustment factors; signals use adjusted prices, fills and costs use raw prices; OCC contract adjustments (splits, special dividends, mergers, spin-offs) are applied by date; adjusted contracts with non-standard deliverables are flagged and excluded by default

Submodule	Responsibility	Details
PIT alignment	Data as of when it was known	Every record carries event_time and available_at; financials effective on announcement date; revisions versioned with as-of queries
IVCalculator and GreeksCalculator	One consistent pricing model	Solve IV from the mid price using the synchronized underlying, rate and dividend, with a European or American model matching the contract style (US equity and ETF options are American; index options such as SPX are European and cash-settled); one time convention (trading or calendar days) everywhere; compute delta, gamma, vega and theta; flag failures instead of guessing
SurfaceBuilder	IV surface per underlying	Grid by delta (or moneyness) × tenor; smoothing and arbitrage checks; derived features such as ATM IV, skew, term structure and IV rank
Universe	Tradable set per day	Underlyings: listing age, liquidity, exclusions. Options: the listed chain at t, filtered by days to expiry, moneyness, spread %, open interest. Expired and delisted contracts are kept in history
Features	Derived data	Dependency graph, cache keyed by parameter hash, incremental updates; features obey available_at like raw data

Submodule	Responsibility	Details
Store	Persistence and versions	Columnar files (for example Parquet) partitioned by date and underlying; a metadata DB for datasets, versions and snapshot IDs
Access	DataView for the layers above	history, cross_section, panel, chain, surface, contract; hot-data cache; a run binds to one data snapshot ID

Four hard rules:

1. No data enters a backtest without an available_at.
2. Cleaning, adjustment and IV calculation are replayable: same raw data and rule version give the same result.
3. An option price and the underlying price used together must come from the same timestamp window.
4. Anything fitted (regime models, IV rank percentiles, surface smoothing, normalization statistics) is fitted only on data before t, using rolling or expanding windows. Regime models report filtered, not smoothed, probabilities, since smoothing uses future data.

Layer 2: Signals

The signal layer answers what to buy or sell and how strongly. How much belongs to the portfolio layer, and whether it can fill belongs to execution.

The signal layer hosts most of the seven strategy components (see Strategy Model): Setup, Entry, Take Profit, Stop Loss, Headwind and position-level Hedge. It is stateful per trade: each component runs only in its lifecycle state, and all of them emit actions from the shared vocabulary. For options, an entry has two parts: a view on the underlying, then the choice of contracts that express it.

Component	Strategy slot	Responsibility	Form
UniverseSelector	—	Pick candidate underlyings	Fixed list, index members, top N by a factor, rule-based screens

Component	Strategy slot	Responsibility	Form
Filter	—	Drop what should not trade now	Liquidity, exclusions, data-quality gates
SetupRule	1 Setup / Regime	Mark a candidate as ready or not	To be defined
EntryRule	2 Entry	Fire the entry while the setup is valid	To be defined
ContractSelector	2 Entry (if needed)	Turn the entry into specific contracts	To be defined
StructureBuilder	2 Entry (if needed)	Define the legs	To be defined
TakeProfitRule	3 Take Profit	Manage gains on an open trade	To be defined
StopLossRule	4 Stop Loss	Manage losses on an open trade	To be defined
HeadwindRule	5 Headwind	Respond to new adverse factors on an open trade	To be defined
PositionHedgeRule	7 Hedge / Protection	Attach or remove protective legs on one trade	To be defined
RollRule	—	Replace legs with a later expiry or new strike	To be defined, optional
SignalCombiner	—	Merge actions issued in the same period	Applies the lifecycle's conflict policy (to be defined)

2.1 Signal output

Field	Meaning
trade_id	The trade this action belongs to; main and hedge legs share it
action	One of the shared vocabulary: OPEN, ADD, REDUCE, CLOSE, ROLL, HEDGE_ADD, HEDGE_REMOVE, SET_STOP, HOLD

Field	Meaning
underlying	The underlying the trade is about
legs	Contract, side, ratio and role (main or hedge) for each leg
fraction	Share of the trade affected, for REDUCE
strength	Optional score or metadata passed on to Positioning
reason	What triggered it, with the data values used
component_id	The component and variant version that produced it

2.2 Two interface styles

- Cross-sectional: score all candidates each period, as in multi-factor stock selection. Input is `cross_section`, output is a set of scored signals.
- Time-series: apply rules to one instrument, as in trend following. Input is `history`, output is that instrument's signal.

Most long-options strategies are time-series rules on the underlying, followed by a contract selector that queries `chain`. Both styles implement the same Component protocol, and the engine treats them alike.

2.3 Boundaries

- Exit, Headwind and Hedge components may read positions (entry premium, holding days, current Greeks, MFE and MAE so far) but never place orders.
- Per-trade decisions live here: stops, targets, headwinds and position-level hedges. Book-wide risk, including total premium, Greeks budgets, book-level headwinds and portfolio hedges, lives in Layer 3.

The full config example, organized by the seven components, is in the Strategy Model section.

Layer 3: Portfolio and Risk

This layer turns signals into target positions and makes sure the portfolio stays within risk limits before any order goes out. Order of work: rebalance check → sizing → optimization (optional) → constraints → portfolio risk → target portfolio.

For a long-options book, the maximum loss on each position is the premium paid. So premium at risk is tracked as a core budget measure, and book risk is aggregated in Greeks. How individual trades are sized is the Positioning slot's job, and its form is not yet decided.

Component	Strategy slot	Responsibility	Form
RebalanceRule	—	Decide whether to act this period	Calendar, drift threshold, signal-driven
Sizer	6 Positioning	Turn OPEN and ADD actions into contract quantities	To be defined
Optimizer	—	Best weights under constraints (optional)	Mean-variance, minimum variance, risk parity
RiskModel	—	Risk estimates for budgets and checks	Greeks aggregation by underlying and expiry bucket, plus beta-weighted delta and vega to a common index, because underlyings such as SPY, QQQ, IWM and their largest holdings are highly correlated; covariance and factor models for the underlyings
Constraint	—	Hard limits	Total premium outstanding, premium per underlying and per expiry, Greeks limits, minimum cash, contract position limits

Component	Strategy slot	Responsibility	Form
RiskRule	5 Headwind (book-wide)	Monitor and intervene on the whole book	Book-wide headwind responses to be defined; general limits such as drawdown breaker and daily loss limit
ScenarioRule	—	Stress the book	Revalue on a grid of spot moves, IV moves and time, with an explicit sticky-strike or sticky-delta assumption; block trades that breach a scenario loss limit
PortfolioHedgeRule	7 Hedge / Protection (book-wide)	Protect the whole book	To be defined; hedge legs tagged with their own trade_id

3.1 Risk actions

When a rule triggers it emits a RiskEvent. Actions are limited to a few kinds so they can be audited:

- Scale down: reduce all positions by a ratio.
- No new positions: only reductions allowed.
- Liquidate: close named contracts or the whole book.
- Pause strategy: until a person or a rule resumes it.

3.2 Output

TargetPortfolioEvent holds the target quantity for each contract plus the reason for every adjustment (which signal, which constraint cut it). Execution turns the gap between target and current positions into orders.

3.3 Who owns which risk

Type	Examples	Layer
Per trade	3 Take Profit, 4 Stop Loss, 5 Headwind on one position, 7 Hedge on one position, rolls	Layer 2 Signals

Type	Examples	Layer
Portfolio	6 Positioning, total premium, Greeks budgets, 5 Headwind on the whole book, 7 Hedge on the whole book, drawdown breaker, scenario limits	Layer 3 Portfolio and Risk

Layer 4: Execution Simulation

Execution decides how much of the target can fill and at what price. Its defaults should lean pessimistic: most of the gap between backtest and live comes from here, and for long options the single biggest source is filling at mid instead of paying the spread.

Component	Responsibility	Details
OrderGenerator	Target − current → orders	Round to whole contracts; sell before buy so cash suffices; split large orders TWAP- or VWAP-style
OrderType	Order behavior	Market, limit, stop, opening or closing auction; DAY or GTC
MarketRule	Reject impossible orders	Trading hours (9:30 to 16:00 ET; some ETF and index options to 16:15), trading halts, Limit Up-Limit Down bands on the underlying, option tick sizes ($0.01 or $0.05 by price and program), position and exercise limits, short-sale borrow availability, T+1 settlement, the pattern day trader rule for small margin accounts
FillModel	Fill price and quantity	Options: buys fill at ask, sells at bid, on the configured snapshot; limit orders fill only when the price trades through; size capped by a participation rate of volume and by displayed size; unfilled remainder carries over or cancels
SlippageModel	Price beyond the quote	Options: a multiple of the half-spread, wider near expiry and for deep out-of-the-money strikes. Underlyings: fixed bps, half-spread, volatility-scaled, square-root impact

Component	Responsibility	Details
CostModel	Explicit fees	Options: per-contract commission plus exchange, ORF and OCC clearing fees, and SEC and FINRA TAF fees on sells. Stocks and ETFs: commission, SEC fee and FINRA TAF on sells, borrow fees on shorts. Also margin interest and funding costs
ExerciseModel	What happens at expiry	Close before expiry (default) or exercise; OCC exercise by exception at expiry (contracts $0.01 or more in the money are auto-exercised unless instructed otherwise); early exercise of American calls before ex-dividend dates; cash or margin needed to take delivery of shares; physical settlement for equity and ETF options, cash settlement (AM or PM) for index options, all into an ExpiryEvent
SimBroker	One interface for the outside	submit, cancel, on_snapshot; emits FillEvent and OrderStatusEvent; the live broker implements the same interface

4.1 Option fill price

A buy fills at the mid price plus a multiple λ of the half-spread. λ = 1 means paying the full ask; test λ = 1.5 to see how much of the edge survives worse fills. Sells mirror this against the bid.

```
\text{fill}_{\text{buy}} = \text{mid} + \lambda \cdot (\text{ask} -
\text{mid}), \qquad \lambda \ge 1
```

For large trades in the underlying, the square-root impact model is a common starting point, where σ is daily volatility, Q is trade size, ADV is average daily volume and k is calibrated from live fills:

```
\text{impact} = k \cdot \sigma \cdot \sqrt{\frac{Q}{\text{ADV}}}
```

4.2 Suggested defaults

- Signal on snapshot t, fill on the quote at t plus the decision latency (for example 15:45 ET signal, 15:50 ET fill), always at bid or ask, never the close.

- Reject entries where the spread exceeds a set % of mid, or open interest is below a minimum.
- Participation cap: start at 5% to 10% of daily contract volume, then run a sensitivity test.
- Close positions before the final days to expiry unless exercise is part of the strategy.
- Log every rejected order and partial fill so research can report their share.

Layer 5: Engine and Accounting

The engine drives events in the right order; accounting records every change in money exactly. Build them separately: scheduling can be complex, bookkeeping must be simple, testable and always correct.

5.1 Engine

Component	Responsibility	Details
EventLoop	Dispatch events	Priority queue ordered by (timestamp, processing phase, sequence) so order is deterministic
Scheduler	Emit timed events	Start of day, snapshot time, rebalance days, end of day, expiry days, all from the trading and expiry calendars
Runner	Manage one run	Load config → bind data snapshot → build components → run → save results; state snapshots for resume
BatchRunner	Run many configs in parallel	Process pool or distributed (for example Ray); an independent random seed per task
VectorizedEngine	Fast screening during research	Matrix operations over all periods with simplified fills; useful for underlying-level signals only

Event-driven is the primary mode for options. Contract selection, rolls and expiry are path-dependent, so vectorized results on option P&L are not trustworthy. Use the vectorized engine to screen views on the underlying, then validate in event-driven mode.

5.2 Accounting

Component	Responsibility	Details
Ledger	Record every money movement	Fills, fees, dividends, interest, exercise and expiry settlements, deposits and withdrawals; append-only
Positions	What is held	Keyed by contract and grouped by trade_id, each leg tagged main or hedge; quantity, multiplier, entry premium, open date, holding days, sellable quantity under the settlement rule
Cash	Cash state	Available, frozen by open orders, margin borrowed
Valuation	End-of-day mark-to-market	Long options marked at bid (conservative), mid also reported; underlyings at close; untradable contracts at last valid price, flagged
CorporateActions	Corporate events	Dividends to cash after tax; splits adjust quantity and cost; contract adjustments update multiplier and strike
Expiry	Settle expiring contracts	Exercise, cash settlement or lapse; removes the position and books the result
PnL	P&L breakdown	Realized, unrealized, fees, dividends; by contract, trade, leg role (main or hedge), the component that closed it, underlying and day
GreeksAttribution	Explain option P&L	Split daily P&L into delta, gamma, vega, theta and residual (see 5.3)
Snapshots	Data for research	Daily equity, cash, positions, premium at risk, net Greeks, turnover

A TradeBook sits beside the ledger: one record per trade_id with its setup, entry, every action and the component that issued it, MFE and MAE in premium and underlying terms, and the final P&L split into main and hedge legs. After a trade closes, its contracts keep being tracked for a set window (shadow tracking), so research can compute post-exit MFE and counterfactual exits. Research reads trades from here.

At every end of day the engine runs the invariant checks from 0.4. If any fails, the run stops and dumps its state.

5.3 Greeks P&L attribution

Each position's daily value change is split using the Greeks from the start of the day, where S is the underlying price, σ is implied volatility and t is time:

```
\Delta V \approx \delta \, \Delta S + \tfrac{1}{2} \gamma \, (\Delta S)^2 + \nu \, \Delta \sigma + \theta \, \Delta t + \varepsilon
```

A large residual ε points to bad data, a jump, or a model mismatch, and is worth investigating.

Layer 6: Research and Reports

Research must answer whether a result is real, not just show a good-looking equity curve. Long-options strategies make this harder: they usually win rarely, earn most of their profit from a few large trades, and pay a persistent volatility risk premium.

Component	Responsibility	Details
ExperimentTracker	Every run findable, comparable, reproducible	run_id, config, commit, data snapshot, summary results, tags; counts how many parameter sets were tried for one idea
ParamSweep	Explore parameters in bulk	Grid, random or Bayesian search; heatmaps show whether results sit on a stable plateau or an isolated spike
Validation	Detect overfitting	In-sample and out-of-sample split; walk-forward (rolling or expanding); purged k-fold; bootstrap of trades (important when trade count is small); one-bar signal delay; doubled costs and λ = 1.5 fills; results split by IV regime

Component	Responsibility	Details
Baselines	Separate skill from structure	Delta-equivalent position in the underlying (does the option add value?); random entries with the same structure, delta and days to expiry (does timing beat the volatility risk premium?)
Metrics	Quantify performance	See 6.1
Attribution	Where returns come from	By contract, underlying and period; Greeks attribution (delta, gamma, vega, theta); contribution of each signal component
Report	Visual output	Equity vs benchmark, drawdowns, monthly returns heatmap, rolling Sharpe, net Greeks over time, premium at risk, IV at entry vs realized volatility, trade list; HTML or notebook export
Compare	Side-by-side results	Runs across parameters or versions; day-by-day backtest vs live gap analysis

6.1 Metrics

Category	Metrics
Return	Cumulative, annualized, excess; return on premium spent
Risk	Annualized volatility, downside volatility, max drawdown, drawdown duration and recovery
Risk-adjusted	Sharpe (read with care for skewed payoffs), Sortino, Calmar, information ratio, Deflated Sharpe
Trade	Win rate, payoff ratio, average holding period, share of profit from the top N trades, turnover, costs as a share of gross P&L, rejected and partial fills
Options	Theta paid, vega P&L, IV at entry vs realized volatility over the holding period, average spread paid as % of premium

Category	Metrics
Capacity	Participation distribution; return decay as capital grows

6.2 Pre-launch checklist

- [ ] Out-of-sample performance does not collapse
- [ ] Results are stable around the chosen parameters
- [ ] Still works with signals delayed by one bar
- [ ] Still positive with doubled costs and $\lambda = 1.5$ fills
- [ ] Profit is not concentrated in a handful of trades or dates
- [ ] Beats the random-entry baseline with the same structure
- [ ] Greeks attribution matches the thesis (a directional strategy should earn mainly from delta)
- [ ] Total number of parameter sets tried is recorded

6.3 Component research

All seven strategy components are researched with the same tools, built once here and reused for every component:

Tool	What it does
Ablation	Run the strategy with one slot switched off or filled by a neutral variant
Variant swap	Fill one slot with a different registered variant, everything else fixed
Slot sweep	Sweep one slot's parameters only, on the same trades where possible
Counterfactual exits	Replay each closed trade under an alternative exit to measure money saved or given up
Attribution by component	Split P&L by the slot whose action opened, sized, reduced, hedged or closed each part of a trade
Lifecycle settings	Compare Open-state run orders and conflict policies with all slots fixed

Each comparison is a pair of runs differing in one component only, recorded in the ExperimentTracker so the history of every change to the strategy stays auditable.

Interaction tests cover dependencies between slots (see Component interactions): vary two slots jointly, for example Take Profit × Stop Loss or Headwind × Hedge, and compare the joint effect with the sum of the single-slot effects. A large gap means the two slots should be researched and tuned together.

6.4 Research protocol

Seven slots, each with variants and parameters, plus lifecycle settings, form a very large search space. With a low-frequency strategy that produces relatively few trades, the main risk is fitting history rather than finding an edge. These rules apply to all component research:

1. Lock a final hold-out period at the start and use it once, at the end.
2. Count every trial across all slots and settings in the ExperimentTracker; the Deflated Sharpe uses the total count.
3. Research in stages, each on walk-forward folds: Setup and Entry first, judged by forward returns of the underlying and of the selected options with no exits applied (event study); then Take Profit and Stop Loss; then Headwind; then Positioning; then Hedge. Finish with interaction tests and a joint re-check.
4. Report confidence intervals, not point estimates: bootstrap by trade and block bootstrap by period. Set a minimum trade count before any conclusion is drawn.
5. Pool trades across underlyings to gain sample size, but require the effect to hold on each underlying separately.
6. Counterfactual exit results are in-sample by construction; any change they suggest must be confirmed out-of-sample before adoption.
7. Compute return and risk metrics from daily mark-to-market P&L of the whole book, not from per-trade returns, so overlapping trades are counted correctly.

Layer 7: Interfaces

The interface layer exposes the system to people and programs, and guarantees that backtest and live trading run the same strategy code with only the data feed and broker swapped.

Component	Responsibility	Details
Python SDK and CLI	Daily entry point for researchers	run, sweep, report, data update; callable from notebooks
REST API	Programmatic access	See 7.1

Component	Responsibility	Details
JobQueue	Run backtests asynchronously	Submit, queue, progress, cancel, timeouts, resource quotas (for example Celery, RQ, Ray)
ResultStore	Keep results	Metadata in a relational DB (for example Postgres); equity, positions and fills in Parquet or object storage
Web UI	Visual operation	Strategy forms generated from parameter schemas, run list, reports, comparisons, data quality dashboard, IV surface and Greeks exposure views
LiveAdapter	Connect to real markets	Real-time underlying and option quote feeds, live broker implementing the SimBroker interface, paper trading, live vs simulated fill reconciliation, kill switch
Auth	Team use	Users and roles, strategy visibility, audit log

7.1 REST API draft

Method	Path	Purpose
POST	/backtests	Submit a config, returns run_id
GET	/backtests/{run_id}	Status, progress, summary metrics
GET	/backtests/{run_id}/equity	Daily equity and drawdown
GET	/backtests/{run_id}/trades	Fill details
GET	/backtests/{run_id}/greeks	Daily net Greeks and Greeks P&L attribution
DELETE	/backtests/{run_id}	Cancel or delete
POST	/sweeps	Submit a parameter sweep
GET	/components	Registered components and their parameter schemas
GET	/datasets	Available datasets and snapshot versions
GET	/chains/{underlying}?date=	Option chain with quotes, IV and Greeks as of a date

7.2 Switching between backtest and live

Module	Backtest	Paper trading	Live
Data feed	Historical snapshot	Real-time quotes	Real-time quotes
Broker	SimBroker	SimBroker	Broker API
Strategy, portfolio and risk components	Same code	Same code	Same code

Directory Structure and Roadmap

Code directories map one-to-one to layers, and dependencies point only downward: an upper layer may import a lower one, never the reverse.

```
backtest/
  core/          events.py  actions.py  component.py  registry.py  clock.py
                 dataview.py  invariants.py  config.py  logging.py
  data/          ingest/  validate.py  clean.py  adjust.py  pit.py
                 universe.py  features/  store.py
    options/     contracts.py  quotes.py  rates.py  iv.py
                 greeks.py  surface.py
  signals/       lifecycle.py  universe/  filters/  combiner.py  strategy.py
    setup/         (1 Setup / Regime variants)
    entry/         (2 Entry variants)
    take_profit/   (3 Take Profit variants)
    stop_loss/     (4 Stop Loss variants)
    headwind/      (5 Headwind variants, per trade)
    hedge/         (7 Hedge / Protection variants, per trade)
    rolls/
  portfolio/     rebalance.py  positioning/ (6 Positioning variants)
                 optimizers/  risk_models/  greeks.py  scenarios.py
                 constraints.py  risk_rules/  hedges/ (7, book-wide)
  execution/     orders.py  market_rules/  fill_models/  slippage/
                 costs/  exercise.py  sim_broker.py
  engine/        loop.py  scheduler.py  runner.py  batch.py  vectorized.py
  accounting/    ledger.py  tradebook.py  positions.py  cash.py
                 valuation.py  corp_actions.py  expiry.py  pnl.py
                 attribution.py  snapshots.py
  research/      tracker.py  sweep.py  validation/  baselines.py
                 ablation.py  counterfactual.py  metrics.py
```

```
                attribution.py  reports/
  api/          sdk.py  cli.py  rest/  jobs/
  live/         feeds/  brokers/  reconcile.py
  tests/        unit/  components/  invariants/  golden/
```

Build order

1. Minimal loop: core events, components, clock and data view; daily underlying data; one entry and one exit rule; equal-weight sizing; next-open fills with a flat fee; accounting and basic metrics. Done when buy-and-hold matches a hand calculation.
2. Credibility: point-in-time data, delisted names, market rules, slippage and impact, invariant assertions, golden regression tests.
3. Options data and execution: contract master, bid and ask snapshots, rates and dividends, IV and Greeks, surface; contract selector and option exit rules; bid and ask fills, option market rules, expiry and exercise; Greeks attribution. Done when a single long call held to expiry matches a hand calculation, including fees and settlement.
4. Strategy components and research: trade lifecycle, action vocabulary, TradeBook, the seven component slots with a neutral variant each, ablation and counterfactual tools; feature cache, vectorized screening, parameter sweeps, walk-forward, baselines, experiment tracking, reports.
5. Portfolio: premium and Greeks budgets, scenario limits, rolls, optional optimizer.
6. Platform: REST API, job queue, web UI.
7. Live: real-time feeds, broker adapter, paper trading, reconciliation.

Credibility and options execution come before research speed, because running faster on an engine you cannot trust produces conclusions you cannot use.

Research and Trade-off Register

Every choice in the system that can be researched or involves a trade-off falls into one of four groups, and the groups are treated differently. Realism assumptions (group 1) are set conservatively and only tested for sensitivity: picking the value that maximizes backtest returns would optimize an illusion. Strategy and research choices (groups 3 and 4) can be optimized, but only under the research protocol in 6.4.

1. Realism assumptions: sensitivity tests only, never optimized

Choice	Trade-off	How to test
Fill price multiple λ (where between mid and ask a buy fills)	Closer to mid gives higher returns but less realism	Run λ = 0.5, 1 and 1.5; the strategy must hold at λ = 1 or worse
Decision latency	Longer is more conservative but may understate what is achievable	Run several latencies (for example 1, 5, 15 minutes) and check how fast returns decay
Participation cap	A higher cap means more capacity but less realism	Returns and capacity at several caps
Mark-to-market at bid or mid	Bid inflates drawdowns and can trigger premium-based stops on spread noise; mid is noisier	Run both and compare stop triggers and drawdowns
Stale-quote tolerance	Stricter rules give cleaner data but fewer usable observations	Several thresholds; conclusions should not change
Data-cleaning rules (for example dropping wide-spread days)	Cleaner data, but may remove stressed periods and bias results toward calm markets	Compare results with and without, focusing on stress periods
Excluding OCC-adjusted contracts	Avoids pricing errors but loses data	Report the share and timing of excluded contracts
American vs European IV model	European is faster; American is accurate for equity and ETF options	Compare IV and Greeks on a sample
Trading-day vs calendar-day time	Changes weekend and holiday theta and IV levels	Pick one everywhere; check Monday P&L attribution
Dividend and rate sources	Forecast vs implied from put-call parity	Compare the effect on in-the-money call pricing
Surface smoothing	More smoothing means less noise but more bias	Check whether smoothed features change Setup decisions
Sticky-strike vs sticky-delta scenarios	The two give different scenario losses	Compute both; risk limits use the more conservative

Choice	Trade-off	How to test
Greeks attribution method (start-of-period vs averaged Greeks)	Different methods leave different residuals	Compare residual size and stability

2. Engineering trade-offs

Choice	Trade-off
Data granularity (daily close vs intraday snapshots)	Cost and storage vs realism; decides how realistic much of the rest can be
check_frequency for Open-state slots	Faster reaction vs more noise and turnover; must equal live trading
Event-driven vs vectorized engine	Accuracy vs speed; vectorized only for underlying-level screening
Underlying universe (index ETFs only vs single stocks)	More trades vs earnings events, wider spreads, borrow constraints and more data
Shadow-tracking window after exit	Fuller counterfactual analysis vs storage and compute

3. Strategy choices to research

Choice	Trade-off
Signal snapshot time	More information closer to the close vs less time before the fill
Contract delta and days to expiry	Leverage and convexity vs theta decay, IV cost and liquidity
Structure (single leg, straddle, spread)	Convexity vs cost; spreads cap upside and need margin
Take Profit approach	Locking in gains vs keeping the tail; much of a long option's value is in the tail, so tight targets erode convexity
Stop Loss on premium vs on the underlying	Premium stops react to IV and spread noise; underlying stops follow the thesis but leave the maximum loss open

Choice	Trade-off
Headwind sensitivity	Over-reaction (false alarms, extra costs) vs under-reaction
Hedge approach and instruments	Protection vs cost, capped upside and basis risk
Rolling	Keeping exposure vs paying the spread again
Positioning (premium budget, scaling in)	Return vs drawdown; scaling in vs entering at once
Regime model	Fast models whipsaw, stable ones lag; more regimes leave fewer samples per regime
Lifecycle settings (run order, conflict policy, concurrency, re-entry)	Each changes strategy behavior and is researched like a component
Book limits (total premium, Greeks caps)	Risk vs opportunity

4. Research-method trade-offs

Choice	Trade-off
Walk-forward window length	Short windows adapt but estimate noisily; long windows the reverse
Hold-out length	More test data means less research data
Pooling across underlyings	Sample size vs differences between underlyings
Number of variants tested	More chances to find a good result vs a larger multiple-testing penalty
Optimization objective	Sharpe penalizes convex payoffs; use Sortino, tail metrics and return on premium alongside it
Minimum trade count and bootstrap block length	Stricter thresholds give more reliable but fewer conclusions

Priorities

For a long-options strategy, five items move results most and should be studied first: λ, decision latency and mark-to-market method from group 1, and contract choice and Take Profit approach from group 3.

Quant Review Notes

A review from a quant research perspective found thirteen issues. All are now handled structurally in this document; six decisions remain open and are listed at the end.

Issue	Why it matters	Resolution	Where
Signal and fill on the same quote	Subtle look-ahead that overstates entry and exit prices	Signal at snapshot t, fill at t plus a decision latency; signals never see that day's close	0.3, 4.2
Stops and targets between snapshots	With snapshot data, the order of a stop and a target hit within one period is unknown	Open-state slots run only at check_frequency, identical to live	Lifecycle settings
Regime models fitted on full history	Smoothed regime probabilities and full-sample percentiles use future data	Fit only on data before t; filtered probabilities	Layer 1, rule 4
Setups vs trades unclear	Concurrent trades and re-entry were undefined	Candidate (per underlying) and trade (per trade_id) scopes; concurrency and re-entry settings	Lifecycle states
Large search space	Seven slots with variants invite false discoveries	Staged research, trial counting, Deflated Sharpe, locked hold-out	6.4
Small, skewed samples	Low-frequency long options give few trades with fat-tailed payoffs	Bootstrap confidence intervals, minimum trade count, pooled but per-underlying checks	6.4

Issue	Why it matters	Resolution	Where
Counterfactual exits lack data	Without prices after exit, stop and target research is guesswork	Shadow tracking of closed contracts	5.2
Correlated underlyings	SPY, QQQ, IWM and their largest holdings behave largely like one position	Beta-weighted delta and vega to a common index	Layer 3 RiskModel
American exercise and dividends	US equity and ETF options are American; a long call can be worth exercising before an ex-dividend date, and ignoring this misstates P&L and expiry handling	Discrete dividend schedule in the data layer; American exercise logic in ExerciseModel; American pricing model for IV	Layers 1 and 4
Inconsistent IV models	Model or time-convention mismatches distort Greeks and attribution	One model per contract style, one time convention, stated sticky assumption	Layers 1 and 3
Metrics on overlapping trades	Per-trade returns misstate book risk	Metrics from daily book mark-to-market P&L	6.4
Adjusted contracts	After splits, special dividends or mergers, OCC-adjusted contracts deliver non-standard baskets; their prices and Greeks no longer match standard pricing	Contract master tracks adjustments; adjusted contracts flagged and excluded by default	Layer 1

Issue	Why it matters	Resolution	Where
Earnings and scheduled events	Single-stock options carry an IV run-up before earnings and a sharp IV drop after; long options held through an event behave very differently	Point-in-time earnings and event calendar in the data layer, readable by Setup, Entry and Headwind; results split by event exposure	Layer 1, Strategy Model, 6.4

Open decisions

Decision	Why it matters
Historical option quote granularity	Intraday bid and ask snapshots are needed for the timing rules; with daily close data only, fills must be modeled more pessimistically and the bias stated
Signal snapshot time and fill latency	Sets how realistic entries and exits are; must match how orders will be placed live
check_frequency for Open-state slots	Determines how quickly stops, targets and headwinds can react, in backtest and live alike
Hold-out period and minimum trade count	Defines when a research result is allowed to count
Hedge instruments	Underlying shares, index futures (ES, MES), options, or a mix; decides what Layers 4 and 5 must support
Underlying universe	Index ETFs only, or single stocks too; single stocks add earnings handling, many more contracts, wider spreads and borrow constraints